

Traction: Innovation Case Study

Submitted By: Saksham Bansal

GitHub Repo: <https://github.com/bansalsaksham/Traction>

The Problem:

OEE is only as good as its data. But capturing downtime reasons at the machine edge is a human-interface problem as much as a software one. The factory floor kills conventional UX.

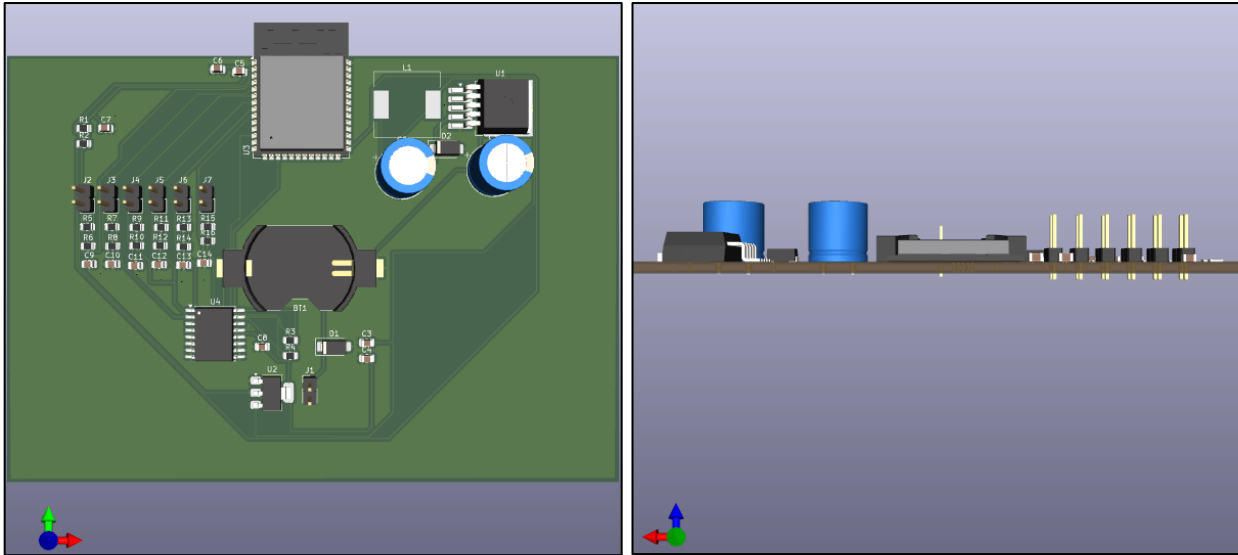
Constraint	Root Cause	Why standard solutions fail
Gloved Hands	Thick Kevlar + metallic dust make capacitive screens useless	Tablet solutions require bare fingers: 15-30s to glove off/on. Fails the 5-second window.
Patchy Wi-Fi	VFDs, metal shielding, and RF interference create 2-hour dead zones	Cloud-first designs lose events silently. Operator never knows. OEE has invisible gaps.
5-Second window	Operator moves to the next task seconds after a machine stops	Any UI requiring navigation or multiple confirmations will simply be abandoned.

Design Philosophy:

1. **Physical First:** Six large 30mm illuminated tactile buttons; one per reason code. No menus, no navigation, no scrolling. One press = one logged event, confirmed by an LED flash, an 85 dB buzzer, and vibration.
2. **Offline First:** Every press is timestamped via a DS3231 RTC and written to an 8 MB LittleFS flash BEFORE any network attempt. When Wi-Fi returns, the device silently replays the queue. The operator never needs to care whether they are online.
3. **OEE Ready:** Each event carries: device_id, machine_id, reason_code, operator_uid (NFC badge tap), UTC timestamp, and a sequence number. Traction-compatible JSON. Sequence gaps are auto detected by the dashboard so any connectivity hole is visible to plant management.

Custom Hardware Implementation (PCB):

To meet the physical constraints of a machine shop, I designed a custom, manufacturing-ready 2-layer PCB in KiCad specifically engineered for high-noise industrial environments.



- **Engineering for the Factory Floor:**

- **Hardware Debouncing (EMI Immunity):** Relying on software debouncing in an environment with Variable Frequency Drives (VFDs) and heavy machinery can lead to false triggers. I designed a dedicated RC hardware debounce matrix ($10\text{k}\Omega/0.1\mu\text{F}$) for all six inputs. This provides a clean, stable signal to the ESP32-S3 before processing occurs, preventing ghost events.
- **Isolated Timekeeping:** The core value of this device is the offline queue. If the timestamp is corrupted, the data is useless. I routed the I2C communication lines for the DS3231 RTC completely isolated from the 150kHz switching node of the onboard buck converter to prevent inductive noise interference.
- **Design for Manufacturing (DFM):** The board was designed for cost-effective production. I overrode the standard ESP32 footprint, expanding the thermal vias from a costly 0.2mm drill size to a standard 0.3mm, reducing fabrication costs without sacrificing thermal performance.
- **RF Optimization:** The board edge is specifically notched to allow the ESP32 antenna to hang off the fiberglass substrate, preventing signal attenuation and maximizing connectivity.

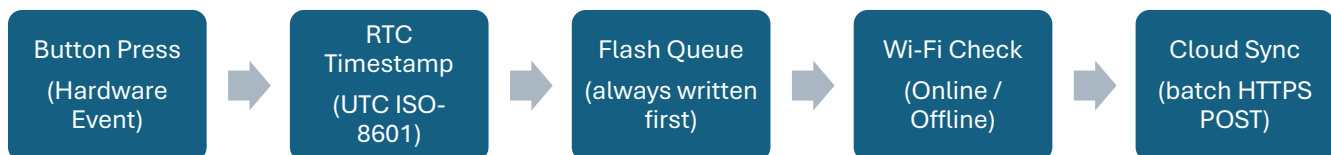
- **Physical Specifications:**

Platform	ESP32-S3 dual-core 240 MHz
Buttons	6 x 30mm illuminated tactile (5N actuation)
Connectivity	Wi-Fi 802.11n + BLE 5.0
Offline Queue	8 MB LittleFS (~65,000 events)
RTC	DS3231 ±2 ppm, coin-cell backup
Power	24 VDC input + 2000 mAh LiPo backup (8h)
Identity	PN532 NFC module

Software & Firmware:

- **Data Flow:** Offline First Architecture

Events are written to flash BEFORE any network attempt. Zero data loss regardless of connectivity.



- **Firmware: Key Patterns**

- **Button ISR with 50 ms hardware debounce:**

```
void IRAM_ATTR isr_btn_0() {  
  uint32_t now = millis();  
  if (!pressed[0] && (now - last[0]) > 50) {  
    pressed[0] = true;  
    last[0] = now;  
  }  
}
```

- **Queue write; always before network attempt:**

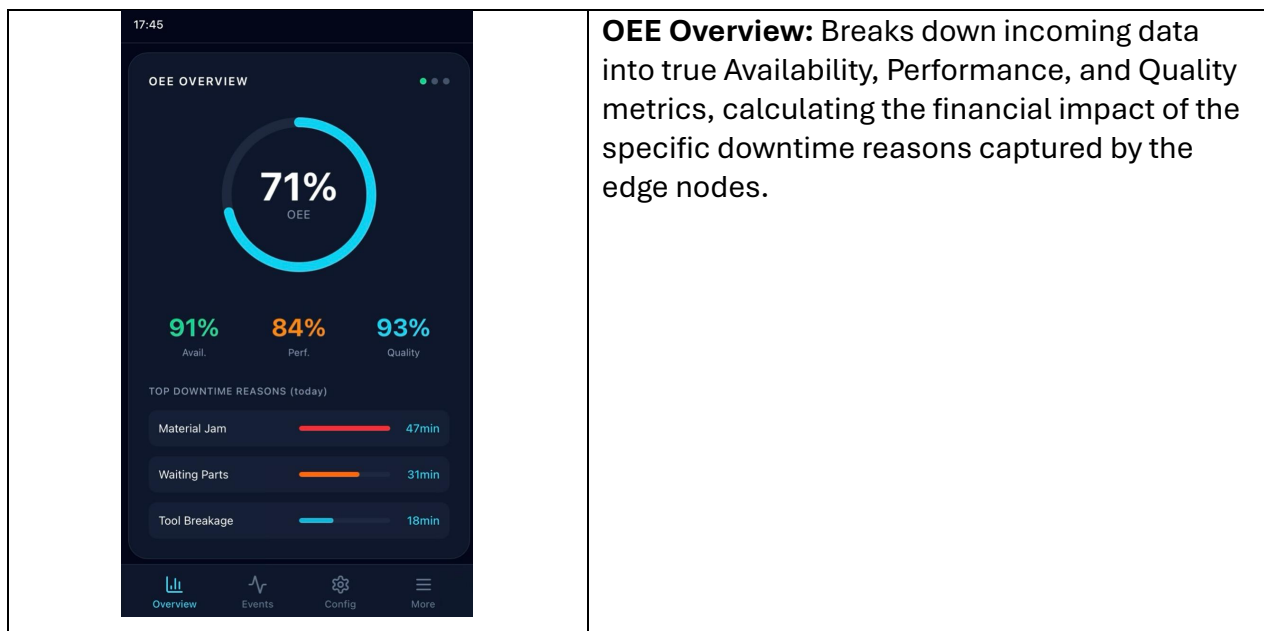
```
queueWrite(seq, rtc.now().unixtime(), buttonIdx); // flash first  
confirmFeedback(buttonIdx); // LED + buzzer + vibration  
if (wifiOnline) syncQueue(); // attempt immediate sync
```

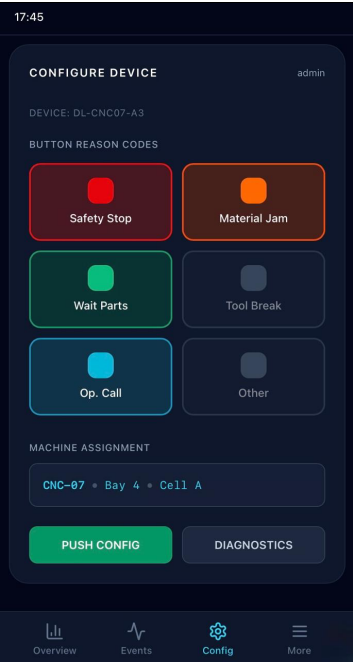
- **JSON event payload schema:**

```
{ "device_id": "DL-CNC07-A3",  
  "seq": 1042,  
  "ts": "2026-04-08T14:22:08Z",  
  "ts_source": "rtc",  
  "reason_code": "MATERIAL_JAM",  
  "reason_label": "Material Jam",  
  "operator_uid": "a3f9d82c",  
  "machine_id": "CNC-07" }
```

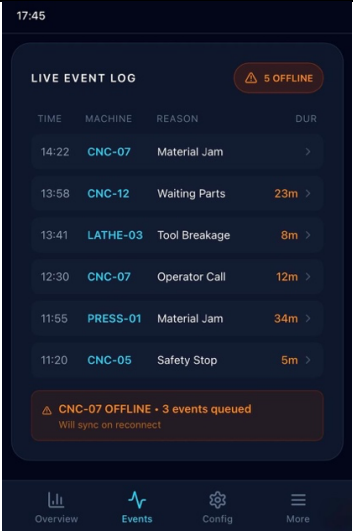
- **Web Dashboard: The Three Screens**

The data collected by the edge node seamlessly populates the Traction software environment, providing plant managers with actionable insights.





Device Configuration: Allows managers to assign color-coded reason codes remotely. This pushes OTA updates to the physical 6-button node on the factory floor, mapping the physical buttons to logical software categories.



Live Event Log: A real-time feed of logged stops. Crucially, offline devices (like CNC-07) are clearly flagged with their queued event counts, proving the system architecture handles 2-hour network drops gracefully.